

How to Take Input From User in Java?

Last Updated : 23 Jul, 2024



Java brings various Streams with its I/O package that helps the user perform all the [Java input-output operations](#). These streams support all types of objects, data types, characters, files, etc. to fully execute the I/O operations. Input in Java can be with certain methods mentioned below in the article.

Methods to Take Input in Java

There are **two ways** by which we can take Java input from the user or from a file

- [BufferedReader Class](#)
- [Scanner Class](#)

1. Using BufferedReader Class for String Input In Java

It is a simple class that is used to **read a sequence of characters**. It has a simple function `read` that reads a character, another `read` which reads an array of characters, and a `readLine()` function which reads a line.

[InputStreamReader](#) is a class that **converts the input stream of bytes** into a stream of characters, allowing it to be read by `BufferedReader`, which expects a stream of characters.

Note: `BufferedReader` can throw checked exceptions.

Below is the implementation of the above approach:

Java



```
1 // Java Program for taking user
2 // input using BufferedReader Class
3 import java.io.*;
4
5 class GFG {
6
```

```

7      // Main Method
8      public static void main(String[] args)
9          throws IOException
10     {
11         // Creating BufferedReader Object
12         // InputStreamReader converts bytes to
13         // stream of character
14         BufferedReader bfn = new BufferedReader(
15             new InputStreamReader(System.in));
16
17         // String reading internally
18         String str = bfn.readLine();
19
20         // Integer reading internally
21         int it = Integer.parseInt(bfn.readLine());
22
23         // Printing String
24         System.out.println("Entered String : " + str);
25
26         // Printing Integer
27         System.out.println("Entered Integer : " + it);
28     }
29 }

```

Output:

```

Mayank Solanki
888
Entered String : Mayank Solanki
Entered Integer : 888

```

Using Buffer Reader Class To Read the Input

Below is the implementation of the above approach:

Java



```
1 import java.io.*;
```

```

2  import java.io.BufferedReader;
3  import java.io.InputStreamReader;
4  class Easy {
5      public static void main(String[] args)
6      {
7          // creating the instance of class
8          // BufferedReader
9          BufferedReader reader = new BufferedReader(
10             new InputStreamReader(System.in));
11         String name;
12         try {
13             System.out.println("Enter your name");
14             name = reader.readLine(); // taking string
15             input
16             System.out.println("Name=" + name);
17         }
18         catch (Exception e) {
19             }
20     }
21 }

```

Output:

```

Enter your name:
Geeks
Name=Geeks

```

2. Using Scanner Class for Taking Input in Java

It is an **advanced version of BufferedReader** which was added in later versions of Java. The scanner can read formatted input. It has different functions for different types of data types.

- The scanner is much easier to read as we don't have to write throws as there is no exception thrown by it.
- It was added in later versions of Java
- It contains predefined functions to read an Integer, Character, and other data types as well.

Syntax of Scanner class:

```
Scanner scn = new Scanner(System.in);
```

Importing Scanner Class:

‘To use the Scanner we need to import the Scanner class from the util package as

```
import java.util.Scanner;
```

Inbuilt Scanner functions are as follows:

- Integer: [nextInt\(\)](#).
- Float: [nextFloat\(\)](#).
- String : [next\(\)](#) and [nextLine\(\)](#)

Hence, in the case of Integer and String in Scanner, [we don't require parsing](#) as we did require in [BufferedReader](#).

Java



```
1 // Java Program to show how to take
2 // input from user using Scanner Class
3
4 import java.util.*;
5
6 class GFG {
7
8     public static void main(String[] args)
9     {
10         // Scanner definition
11         Scanner scn = new Scanner(System.in);
12
13         // input is a string ( one word )
14         // read by next() function
15         String str1 = scn.next();
16     }
```

```

17         // print String
18         System.out.println("Entered String str1 : " +
    str1);
19
20         // input is a String ( complete Sentence )
21         // read by nextLine()function
22         String str2 = scn.nextLine();
23
24         // print string
25         System.out.println("Entered String str2 : " +
    str2);
26
27         // input is an Integer
28         // read by nextInt() function
29         int x = scn.nextInt();
30
31         // print integer
32         System.out.println("Entered Integer : " + x);
33
34         // input is a floatingValue
35         // read by nextFloat() function
36         float f = scn.nextFloat();
37
38         // print floating value
39         System.out.println("Entered FloatValue : " +
    f);
40     }
41 }

```

Output:

```

Entered String str1 : Geeks
Entered String str2 : Geeks For Geeks
Entered Integer : 123
Entered FloatValue : 123.090

```

Example 2:

Java



```
1 // Java Program to implement
2 // Scanner Class to take input
3 import java.io.*;
4 import java.util.Scanner;
5
6 // Driver Class
7 class Easy {
8     // main function
9     public static void main(String[] args)
10    {
11        // creating the instance of class Scanner
12        Scanner obj = new Scanner(System.in);
13        String name;
14        int rollno;
15        float marks;
16
17        System.out.println("Enter your name");
18
19        // taking string input
20        name = obj.nextLine();
21        System.out.println("Enter your rollno");
22
23        // taking integer input
24        rollno = obj.nextInt();
25        System.out.println("Enter your marks");
26
27        // taking float input
28        marks = obj.nextFloat();
29
30        // printing the output
31        System.out.println("Name=" + name);
32        System.out.println("Rollno=" + rollno);
33        System.out.println("Marks=" + marks);
34    }
35 }
```

Output:

```
Enter your name
Geeks
```

Enter your rollno

5

Enter your marks

84.60

Name=Geeks

Rollno=5

Marks=84.60

Differences Between BufferedReader and Scanner

- BufferedReader is a very basic way to read the input generally used to read the stream of characters. It gives an edge over Scanner as it is **faster than Scanner** because Scanner does lots of post-processing for parsing the input; as seen in `nextInt()`, `nextFloat()`
- BufferedReader is more flexible as we can specify the size of stream input to be read. (In general, it is there that BufferedReader reads larger input than Scanner)
- These two factors come into play when we are reading larger input. In general, the Scanner Class serves the input.
- BufferedReader is preferred as it **is synchronized. While dealing with multiple threads it is preferred.**
- For decent input, and easy readability. The Scanner is preferred over BufferedReader.

Input in Java

[Visit Course ↗](#)

 Comment

More info 

Next Article >

Java if statement

Similar Reads

Java Tutorial

Java is one of the most popular and widely used programming languages which is used to develop mobile apps, desktop apps, web apps, web servers, games, and enterprise-level systems. Java was...

🕒 4 min read

Java Overview



Java Basics



Java Syntax

Java is an object-oriented programming language which is known for its simplicity, portability, and robustness. The syntax of Java programming language is very closely aligned with C and C++ which...

🕒 5 min read

Java Hello World Program

Java is one of the most popular and widely used programming languages and platforms. Java is fast, reliable, and secure. Java is used in every nook and corner from desktop to web applications, scientific...

🕒 5 min read

Java Identifiers

An identifier in Java is the name given to Variables, Classes, Methods, Packages, Interfaces, etc. These are the unique names and every Java Variables must be identified with unique names. Example: public...

🕒 2 min read

Java Keywords

In Java, Keywords are the Reserved words in a programming language that are used for some internal process or represent some predefined actions. These words are therefore not allowed to use as variabl...

🕒 5 min read

Java Data Types

Java is statically typed and also a strongly typed language because, in Java, each type of data (such as integer, character, hexadecimal, packed decimal, and so forth) is predefined as part of the programmin...

🕒 11 min read

Java Variables

Variables are the containers for storing the data values or you can also call it a memory location name for the data. Every variable has a: Data Type - The kind of data that it can hold. For example, int, string,...

🕒 8 min read

Scope of Variables In Java

Scope of a variable is the part of the program where the variable is accessible. Like C/C++, in Java, all identifiers are lexically (or statically) scoped, i.e. scope of a variable can be determined at compile time...

🕒 5 min read

Java Operators

Java operators are special symbols that perform operations on variables or values. They can be classified into several categories based on their functionality. These operators play a crucial role in performing...

🕒 14 min read

How to Take Input From User in Java?

Java brings various Streams with its I/O package that helps the user perform all the Java input-output operations. These streams support all types of objects, data types, characters, files, etc. to fully execut...

🕒 5 min read

Java Flow Control



Java Methods



Java Arrays



Java Strings



Java OOPs Concepts



Java Interfaces



Java Collections



Java Exception Handling



Java Multithreading



Java File Handling



Java Streams and Lambda Expressions



Java IO



Java Synchronization



Java Regex



Java Networking



JDBC



Java Memory Allocation



Java Interview Questions



Java Practice Problems



Java Projects



Article Tags :

Java

java-basics

Practice Tags :

Java

